

Task 1: Schema Design and Implementation

Proposed Schema

Hybrid Schema Design

The schema is designed as a hybrid of Star Schema and denormalization, leveraging BigQuery's columnar storage, partitioning, and compression techniques to optimize analytics performance. This approach balances query efficiency, storage optimization, and data integrity.

Schema Overview

Fact Table (**fact_orders**):

1. Denormalized to include key transactional details such as order_id, amount, currency, and order_status.
2. Reduces the need for frequent joins with related tables, improving query performance for metrics aggregation.

Dimension Tables:

dim_customers:

1. Normalized to store descriptive and relatively static customer attributes like name and company.

dim_addresses:

1. Normalized to store address details (street, city, country) for both billing and shipping addresses.
2. Normalization reduces redundancy in dimension data, which BigQuery can compress efficiently using columnar storage techniques like run-length encoding.

```
-- Fact Table
CREATE TABLE project_id.dataset_id.fact_orders (
  order_id STRING,
  amount FLOAT64,
  currency STRING,
  order_status STRING,
  customer_id STRING,
  billing_address_id STRING,
  shipping_address_id STRING,
  payment_method STRING,
  payment_status STRING,
  shipping_status STRING,
  order_date DATE
)
PARTITION BY DATE(order_date)
CLUSTER BY currency, country;
```

```
-- Dimension Table: Customers
CREATE TABLE project_id.dataset_id.dim_customers (
  customer_id STRING PRIMARY KEY,
  name STRING,
  company STRING
);
```

```
-- Dimension Table: Addresses
CREATE TABLE project_id.dataset_id.dim_addresses (
  address_id STRING PRIMARY KEY,
  name STRING,
  street STRING,
  zip_code STRING,
  city STRING,
  country STRING
);
```

Advantages of the Hybrid Approach

Efficient Query Performance:

1. Denormalized fact tables reduce the need for frequent joins during metrics aggregation (e.g., total revenue by country).
2. Partitioning ensures that only relevant partitions are scanned for time-based queries, minimizing data reads.

Storage Optimization:

1. Normalized dimension tables reduce redundancy in static descriptive data (e.g., customer details), which BigQuery compresses efficiently using columnar storage techniques.

Scalability:

1. Partitioning and clustering allow the schema to scale with increasing data volumes while maintaining query performance.

Cost Efficiency:

1. Compression reduces storage costs without compromising query speed.
2. Selective column scanning minimizes the amount of data processed during queries.

Disadvantages of the Hybrid Schema

Increased Complexity in ETL Process:

1. Combining denormalized fact tables with normalized dimension tables requires additional transformations during ingestion, potentially increasing pipeline complexity.

Storage Overhead in Fact Table:

2. The denormalization of fact_orders may result in redundant storage for some attributes (e.g., customer_id, billing_address_id, shipping_address_id), especially for repeated customers or addresses.

Dependence on Dimension Tables:

1. While denormalization reduces some joins, queries still require joins with dimension tables (dim_customers, dim_addresses) for descriptive information. This may slightly impact query performance for complex queries.

Sample Query

1. Tracking Revenue Trends by Country and Currency:

```
SELECT
  EXTRACT(YEAR FROM fo.order_date) AS year,
  EXTRACT(MONTH FROM fo.order_date) AS month,
  da.country,
  fo.currency,
  SUM(fo.amount) AS total_revenue
FROM project_id.dataset_id.fact_orders fo
JOIN project_id.dataset_id.dim_addresses da
  ON fo.billing_address_id = da.address_id
```

```
GROUP BY
    year,
    month,
    da.country,
    fo.currency
ORDER BY
    year,
    month,
    da.country,
    fo.currency;
```

2. Analyzing Customer Activity and Order History:

```
SELECT
    c.customer_id AS customer_id,
    c.name AS customer_name,
    COUNT(fo.order_id) AS total_orders,
    SUM(fo.amount) AS total_spent,
    AVG(fo.amount) AS average_order_value,
    MIN(fo.order_date) AS first_order_date,
    MAX(fo.order_date) AS most_recent_order_date,
    COUNT(CASE WHEN fo.order_status = 'Delivered' THEN 1 END) AS
delivered_orders,
    COUNT(CASE WHEN fo.order_status = 'Pending' THEN 1 END) AS pending_orders,
    COUNT(CASE WHEN fo.order_status = 'Cancelled' THEN 1 END) AS
cancelled_orders
FROM project_id.dataset_id.fact_orders fo
JOIN project_id.dataset_id.dim_customers c
    ON fo.customer_id = c.customer_id
GROUP BY
    c.customer_id,
    c.name
ORDER BY
    total_spent DESC;
```

3. Monitoring shipping performance:

```
WITH zip_code_performance AS (  
    SELECT  
        da.zip_code,  
        da.city AS destination_city,  
        da.country AS destination_country,  
        COUNT(*) AS total_orders,  
        COUNT(CASE WHEN fo.shipping_status = 'Delivered' THEN 1 END) AS  
delivered_orders,  
        COUNT(CASE WHEN fo.shipping_status != 'Delivered' THEN 1 END) AS  
not_delivered_orders,  
        ROUND(  
            100.0 * COUNT(CASE WHEN fo.shipping_status = 'Delivered' THEN 1 END)  
/ COUNT(*),  
            2  
        ) AS delivery_success_rate  
    FROM project_id.dataset_id.fact_orders fo  
    JOIN project_id.dataset_id.dim_addresses da  
        ON fo.shipping_address_id = da.address_id  
    GROUP BY da.zip_code, da.city, da.country  
)  
SELECT *  
FROM zip_code_performance  
ORDER BY delivery_success_rate ASC, total_orders DESC;
```

Considerations for Data Correctness

Data Validation During Ingestion:

1. Ensure primary keys (e.g., order_id, customer_id) are unique and non-null.
2. Validate foreign key relationships between the fact table and dimension tables.

Handling Anomalies:

1. Detect negative or zero values in numeric fields like amount.

-Handling the return orders

```
SELECT * FROM project_id.dataset_id.fact_orders WHERE amount <= 0;
```

2. Monitor missing or inconsistent data (e.g., missing addresses or invalid currencies).

Data Updates:

1. Use BigQuery's MERGE statement to handle updates or corrections in source data efficiently.

Task 2: Data Pipeline and Quality Assurance

This solution leverages Google Cloud Pub/Sub, Google Cloud Storage (GCS), and BigQuery to create a scalable and robust pipeline. The design focuses on simplicity, minimal transformations, and the inclusion of audit fields for tracking data lineage and ensuring data quality.

Pipeline Design

Google Cloud Pub/Sub:

1. Acts as the messaging layer to decouple the ERP system's API from downstream components.
2. The ERP system publishes data (e.g., orders, customers, shipments) to specific Pub/Sub topics.
3. Subscriptions are created to pull messages from these topics and write them to Google Cloud Storage.

Google Cloud Storage (GCS):

1. GCS acts as a staging area for raw data before ingestion into BigQuery.
2. Stores raw data in a structured format (e.g., JSON or Avro) for durability and easy integration with BigQuery.
3. Files are organized in a structured folder hierarchy for easy management:
4. Supports integration with BigQuery for efficient batch loading or near real time data loading

```
gs://koro_datalake/raw_data/orders/YYYY/MM/DD/  
gs://koro_datalake/raw_data/customers/YYYY/MM/DD/
```

BigQuery:

1. Ingests raw data directly from GCS into a raw_data dataset.
2. Minimal transformations are applied during ingestion, focusing on schema validation and adding audit fields.
3. Data is organized into tables designed for analytics-ready use cases.

Data Quality Assurance

Validation During Ingestion:

1. Enable schema validation during BigQuery load jobs to ensure data conforms to predefined schemas.
2. Apply schema validation during BigQuery ingestion to reject invalid rows

Deduplication:

1. Use unique keys (e.g., order_id) to identify duplicate records during ingestion.

Anomaly Detection:

1. Run scheduled SQL queries in BigQuery to detect anomalies such as negative amounts or invalid statuses. Example query:

```
SELECT *  
FROM raw_data.fact_orders  
WHERE amount < 0 OR order_status NOT IN ('Pending', 'Completed',  
'Cancelled');
```

Monitoring and Alerts:

1. Use Cloud Monitoring and Logging to track pipeline performance and set up alerts for failures or anomalies.

Strategies to Ensure Data Security Considerations (GDPR and PII Compliance)

GDPR Compliance:

1. Minimize Personally Identifiable Information (PII), Payment Card Industry (PCI) data, and Protected Health Information (PHI) storage; pseudonymize sensitive fields when possible.

Encryption:

1. Encrypt data at rest (GCS/BigQuery) and in transit using Google-managed encryption keys.

Access Control and Audit Logging:

1. Implement Role-Based Access Control (RBAC)
2. Enable audit logs for Pub/Sub, GCS, and BigQuery to track access and changes.

Task 3: ELT with dbt and BigQuery Projects

dbt (Data Build Tool) is central to the ELT process, providing a modular and scalable framework for transforming raw data into analytics-ready models. Here's how dbt operates within the described structure:

Choose between dbt Core for custom CI/CD pipelines or dbt Cloud for a managed, user-friendly experience with scheduling and monitoring.

Workflow in BigQuery with dbt

Raw Data in BigQuery:

1. Raw data from the ERP system is ingested into the raw data project (e.g., raw_orders, raw_customers tables).
2. dbt accesses this raw data and applies transformations to create cleaned, standardized models.

Staging Models:

1. dbt creates staging models that:
2. Clean and normalize raw data.
3. Perform light transformations (e.g., handling null values, standardizing currencies).
4. Join related tables to enrich data for downstream use.
5. Staging models are materialized as views for flexibility during development.

Production Models (Marts):

1. dbt builds fact and dimension tables (e.g., fact_orders, dim_customers) from staging models.
2. These models are materialized as tables in the production project, optimized for reporting and analytics.
3. Business logic is encoded in SQL transformations, ensuring consistency across teams.

Deployment to Production:

1. Once tested in the sandbox, dbt models are deployed to the production project using dbt run --target production.

Testing and Quality Assurance:

1. dbt's built-in testing framework validates models (e.g., unique keys, not-null constraints).
2. Custom tests ensure business logic is accurate.

Version Control and Documentation:

1. dbt integrates with Git for version control, ensuring traceability of changes.
2. A **peer review** process is enforced via GitHub branch protection rules. At least one approval is required from a teammate before merging.
3. dbt auto-generates model documentation, including descriptions, dependencies, and lineage.

dbt Project Structure

```
dbt_project/
├── dbt_project.yml
├── models/
│   ├── raw/
│   │   ├── raw_orders.sql
│   │   ├── raw_customers.sql
│   │   ├── raw_shipments.sql
│   │   ├── raw_billing.sql
│   │   └── raw_addresses.sql
│   ├── staging/
│   │   ├── stg_orders.sql
│   │   ├── stg_customers.sql
│   │   ├── stg_shipments.sql
│   │   ├── stg_billing.sql
│   │   ├── stg_addresses.sql
│   │   └── stg_customer_orders.sql # Joins and cleans combined data
│   └── marts/
│       ├── analytics/
│       │   ├── fact_orders.sql
│       │   ├── fact_customers.sql
│       │   └── fact_shipping.sql
│       └── dashboards/
│           ├── revenue_by_country.sql
│           ├── customer_activity.sql
│           └── shipping_performance.sql
├── tests/
│   ├── schema_tests.yml
│   └── custom_tests.sql
├── macros/
│   └── custom_macros.sql
├── seeds/
│   └── static_data.csv
```

dbt_project.yml

```
# Name of the dbt project
name: 'koro_dbt_project'

# Version of the project
version: '1.0.0'

# Define the configuration version
config-version: 2

# Specify the database connection profile defined in profiles.yml
profile: 'koro_bigquery_profile'

# Paths for dbt operations
target-path: "target" # Directory where compiled SQL and manifest.json will be
                        # stored
clean-targets:
  - "target"
  - "dbt_modules"

# Model configurations
models:
  # Root directory for models
  koro_dbt_project: # Base directory of the dbt project
    raw:
      # Configurations for raw models
      +schema: 'raw'
      +materialized: 'view' # Raw data is typically stored as views
    staging:
      # Configurations for staging models
      +schema: 'staging'
      +materialized: 'view' # Staging models are typically views for easy
updates
    marts:
      # Configurations for marts (finalized, denormalized tables)
      +schema: 'marts'
      +materialized: 'table' # Finalized models stored as tables for optimized
queries
      +tags: ['analytics', 'dashboard'] # Tagging marts models for analytics
use
      +grant_access_to:
        - project: analytics_project
          dataset: analytics_dataset
        - project: bi_tools_project
          dataset: bi_tools_dataset

# Seed configurations
seeds:
```

```

default:
  +schema: 'static'
  +quote_columns: false

# Snapshot configurations for maintaining time-variant data
snapshots:
  default:
    +schema: 'snapshots'

# Test configurations for schema and data integrity
tests:
  default:
    +schema: 'test'

```

Profile.yml

Sandbox dbt schema will be created for each user based on their userid.

```

koro_dbt_project:
  target: sandbox
  outputs:
    # Production target: Used for finalized models, materializing into the
    # production project.
    production:
      type: bigquery
      method: service-account
      keyfile: "{{ env_var('DBT_KEYFILE_PATH') }}"
      project: koro-production
      dataset: production
      threads: 10
      priority: interactive
      location: europe-west1 # Replace with your preferred BigQuery region

    # Sandbox target: Used for development and testing.
    sandbox:
      type: bigquery
      method: oauth # Personal OAuth for developer-specific workspaces
      project: koro-sandbox
      dataset: "dev_{{ env_var('USER').replace('.', '_') }}" # Creates
      # user-specific dev schemas
      threads: 4
      priority: interactive
      location: europe-west1 # Replace with your preferred BigQuery region

```

Authorised Views for Restricted Access using dbt:

1. Using dbt, you can create authorised views to share specific datasets with other teams while restricting access to sensitive fields and its mentioned in the project.yml
2. Grants direct access to specific projects and datasets (e.g., analytics_project.analytics_dataset and bi_tools_project.analytics_dataset) for marts models.

```
{{ config(
    materialized='view',
    grant_access_to=[
        {'project': 'production_project', 'dataset': 'analytics'}
    ]
) }}
```

```
SELECT
    order_id,
    amount,
    currency,
    DATE(order_date) AS order_date
FROM {{ source('raw_data', 'raw_orders') }}
WHERE order_status = 'Completed';
```

Proposed IAM Roles and Permissions for Security

Role	IAM Role	Access Level
Engineers	<code>roles/bigquery.dataEditor</code>	Full access to query, insert, update, and delete raw tables and views in the <code>raw_data</code> project.
Analysts & Developers	<code>roles/bigquery.dataViewer</code>	Read-only access to raw tables for occasional reference, if allowed.
Developers	<code>roles/bigquery.dataEditor</code>	Full access to create, modify, and delete datasets, tables, and views in the <code>sandbox</code> project.
Analysts	<code>roles/bigquery.dataViewer</code>	Read-only access to analytics-ready models in the <code>production</code> project for reporting purposes.
Visualization Tools	<code>roles/bigquery.dataViewer</code>	Read-only access to production datasets for dashboards and analytics tools like Looker/Tableau.